

4 Day3 Notes - Agentic AI

What is Agentic AI?

It refers to the AI system that can act like intelligent agents, the agents which respond to inputs, but can:

Reason

Plan

Make decisions

Take action

Use tools

Remember past interactions

And most importantly, operate autonomously towards a goal

What is traditional AI system?

Traditional AI is what we have seen in the past decade:

A chatbot that answers questions

A model that classifies images

A tool that summarises text

These are narrow AI systems because they are task specific, they are stateless, which means there is no memory, they are reactive. They do one thing well., when asked.

Example:

You ask ChatGPT to summarise an article

It reads your input generates a summary, and that's it

It does not remember your previous question

It does not decide what to do next

It does not ask: should I find more details from the web? or Agenticity it is smart, but it is not Agentic.

It is smart, but it is not Agentic

How is Agentic AI different?

Feature	Traditional AI	Agentic AI
Reactive or proactive	Reactive	Proactive
Has memory	X	✓
Plans actions	X	✓
Take decisions	X	✓
Tool use	No (manual)	✓ (automatically)
Goal oriented behaviour	X	✓
Multistep reasoning	Hard / absent	✓
Example examples	Img classifiers, Sentiment analyzers, basic chatbots	AutoGPT, ChatGPT Plugin, Langchain agents, Copilots.

Real world, examples of Agentic AI:

1. Github copilot.

Instead of just suggesting code, it understands what you are trying to build.

It helps write whole functions

It learns from previous context

2. Customer support agent.

One Agent handles customer issue logging

Another agent checks the backend for the order status

Third agent offers a resolution - all without human input

Components of AI agents

As I told you an intelligent Agent needs more than just knowledge. It needs to remember, think, plan and do. - just like a human assistant.

There are four key parts that make an AI system, truly Agentic

1. Memory -, remembering what happened before.

Memory allows the agent to store the past interactions, actions, and results, so it can use that knowledge in the future.

Langchain has a memory module called as ConversationBufferMemory - it's Stores dialogue history for natural conversations.

2. Reasoning - thinking through a situation

It lets Agent analyse the information, make logical decisions, and solve problems.

Example:

Let's say your goal is: plan a trip to Manali

If the flights are too costly, you might reason: let me check trains instead

If the hotels are overbooked, you look for nearby hotels.

This is what is called as reasoning - not just reacting, but thinking through the options.

Auto GPT uses reasoning to decide:

First, search Google for best places

Then, summarise each

Finally, decide which one matches user preference.

Langchain uses ReAct (Reasoning + Acting) framework to let agents think before doing.

Auto GPT uses chain of thought reasoning loop.

3. Planning - Breaking down a goal into steps

Planning helps the agent create a step-by-step path from goal to result.

It is like saying:

Goal: write a blog post on AI agents

Plan: 1. Research AI agents. 2. Write a draft. 3. Review and edit. 4. Publish.

So we clearly understand that, without a plan, the agent would just guess what to do next.

4. Actions - doing something in the world

This is the part where the agent execute the tasks like:

Search searching the web

Sending emails

Running ~~courses~~ *code s*

Fetching data from APIs

Action = execution

Langchain agent uses a Google search API to fetch latest data.

Agent can also use python code to calculate financial metrics

Auto GPT using a browser to click links or download the data.

How do these four work together?

Imagine you gave an AI Agent a goal:

"find the best laptop under ₹60,000 and email me a summary."

1. Memory.

It remembers your budget and past preferences(let's say in the past, you liked Dell laptops)

2. Reasoning

It thinks: "I should search Amazon and Flipkart. But first, let me define what 'best' means? - maybe battery life or maybe performance or maybe both."

3. Planning.

a. Search top laptops.

b. Compare specifications.

c. Filter under ₹60,000.

d. Write summary.

e. Send email.

4. Actions.

Execute each step using APIs and tools like web search and email integration

Framework for building AI agents

It is a python framework designed to build LLM powered application applications - especially those that need memory, tools, reasoning, agents.

How langchain enable Agentic AI

1. Memory: ConversationBufferMemory, VectorStoreRetrieverMemory, etc

2. Reasoning: ReAct (Reason + Act) agent architecture

3. Planning - Sequential Chains, tool-use decision trees

4.Action: Tool integrations: Google search, APIs, web scrapping, zapier, etc.

Some of the real world use cases that you can solve using this framework Car:

1. Building customer support bots that pull knowledge from documents.

2. Finance agents that summarise reports and call APIs

3. Research assistance that search read and answer questions.

4. Autonomous coders that plant software projects.

2. AutoGPT - fully autonomous goal driven Agent

This is an open source, python application that wraps GPT4 and turn it into a fully autonomous Agent.

You just give it the goal, and auto GPT:

- Break it down into sub tasks

- Things through steps

- Uses tools like web search, file saving, APIs

- Keeps looping until it reaches the goal or fails and retries.

It is like giving GPT for a to-do list, a web browser, and task manager - and letting it run on its own

Core features:

- Recursive self prompting: it keeps talking to itself until tasks are done

- Tool usage - built-in Internet search, file writing, code, execution

- Memory - stores results and uses them in future steps.

Latest, consider a real world use case with a goal: create a market analysis of AI start-ups

- AutoGPT will search Google for news

- It will extract and summarise the data

- Write a report

- Save it locally.... And all of this done autonomously.

3. BabyAGI - task execution, Agent with planning

It is a simplified, Agent that:

Which framework to choose in which situation?

- Takes a goal

- Maintains a task list

- Plans and re-prioritises tasks

- Uses GPT 4 to complete each task

- Update the list with new tasks if needed

How it works

1. Get a goal.: "grow my Twitter following"

2. Create tasks.:

- research trending topics

- write a tweet

- schedule it

3. After each task, GPT4 evaluate the result and create a new task if needed.

4. Repeats until goal is done.

Some of the use cases where this can be used are:

- Content creation automation

- Social media planning

- Long-term project tracking with AI help

them.laramework to choose in which situation?

Use langchain if you want to build production ready agents and have control over the steps, tools and logic.

This is like a developer kit for building custom, AI workflows and agents.

You can use it when you want to integrate AI into the system.

You can use it when you want to define exactly how the agent thinks and acts.

You can use it when you care about modularity, reliability and long-term use.

Example:

Healthcare assistant to help doctors, query, patient history, and summarise lab reports. This is referred using this framework because custom PDF readers are needed., we may need to use RAG to query cases, we may need to connect to secure hospital database, we may need structured error handling—langchain gives us the total control over the tool use, memory, data security, and output format.

You can also use it as a finance report generator where the agent reads balance sheets generate summaries and emails them.langchain lets you:

- Integrate with google drive
- uses GPT 4 for summarisation
- runs custom calculations using python
- trigger email alerts using zapier

2. Use auto GPT when you want to automate and entire goal with minimum coding and you are okay with.:

- the agent, figuring out the steps by itself
- less control over each step
- experimenting with open and task

Some of the real life examples of this could be:

a. Market research agent.

Goal: analyse top five electric vehicle start-ups in the US and create a report

Auto GPT:

- google, search the companies
- read websites
- summarise, strengths, and weaknesses
- save a report as a text or PDF file.

There is no need to define tools steps as it figures out.

b. Another good use case can be e-book generator..

Some important things to note for this framework:

- can be unpredictable
- may loop indefinitely
- need access to APIs, browser, or OS.

You can think of using this framework when:

- you want full automation of vague goals
- you don't want to hand Code logic
- you are happy with experimentation
- MVPs and one time tasks

3. Use BabyAGI if you want to manage dynamic task lists and observe how an AI:

- create new tasks
- reprioritise them
- execute them step-by-step

Of the real life use cases where this can be used:

a. Social media manager, Agent.:

Goal-increase Instagram engagement for my brand

BabyAGI:

- create tasks-research competitors, generate content, schedule posts
- finishes, one task
- evaluate the result
- add new task, if needed
- keeps improving over time

b. Business growth planner.

So overall, you can think of using this framework when:

- you want task tracking with AI
- you want Agent to create and adjust tasks
- you want dynamic goal refinement
- your objective is for long running and evolving tasks

4. AutoGen

This framework is developed by Microsoft, and it lets you to build multi Agent conversations between the LLMs, tools, and users.

Unlike auto GPT, this framework allows multiple specialised agents to talk to each other and solve problems together.

Each agent can:

- have its own role
- use tools or run code
- collaborate with others via structured chats

Be autonomous or human in the loop

When should you use this framework?:

Use it if you want to:

- build complex, Agent workflows
- simulate, real world team collaboration
- assign different skills or roles to agents
- let agents debate, validate, or approve each other's ideas
- combine human and LLM in a looped decision process

